Wholesaler Territory Division — Handoff

Read this file first. It is written for a fresh context window.

Goal: divide a shared sales territory between two merging wholesaling forces, fairly, at the postal-code level. Model, theory and reference implementation are settled. The next step is a production Python implementation against real ZCTA data.

Updated 2026-08-27 to reflect the adversarial review's reversal of the fairness baseline (`review/HANDOFF_REVIEW.md` §2: the disagreement point is `d=(0,0)`, not pre-merger production) and the synthetic battery (`battery/FINDINGS.md`). The code in `code/` implements `d=(0,0)`; gains are bundle utilities.

Same-day addendum. Battery figures now render as filled zip-level maps (Voronoi choropleth, matching `mkfig_census.py`'s style) instead of node-link scatter plots. `synth.py` gained a heavy-tailed sales-value dial (double Pareto-lognormal; see `code/TAIL_DISTRIBUTION_NOTE.md`) and a ninth battery case, **C9**, run against it: contiguity non-convergence turns out to have (at least) three independent causes, not one — see §3 trap 11 and §6.

---

1. Decisions already made — do not relitigate

Decision	Choice	Why
Utility form	**Linear / additive** in the three fields	Cobb-Douglas was tried and rejected: it collapsed to a monotone function of one variable, and its zero-pathology (a rep with no inherited book gets ~zero utility) is fatal on real data
Headroom convention	**Net** — residual after own sales and the *captured* share of the competitor's	Gross makes inherited business a *liability* whenever `λ > 0`, which is incoherent
Solution concept	**Maximum Nash welfare** (max `g_a·g_b` at `d=0`)	Exactly solvable (prefix property), Pareto efficient by construction; EF1 is a theorem at the zero baseline (Caragiannis et al. 2019) and holds *only* there — see `review/HANDOFF_REVIEW.md` §2

Decision	Choice	Why
Fairness baseline	**`d=(0,0)`** — gains are bundle utilities	REVERSED from pre-merger production by the review: a Nash disagreement point must be unilaterally guaranteeable, and after the merger neither rep can revert to their legacy book. The pre-merger baseline is equivalent to an implicit, unstable asymmetric-Nash weight (omega≈0.54) and forfeits EF1. Zero also restores the optimal-transport correspondence. Applies to **all** criteria in the code (KS, egalitarian, equal-gain included) — the paper's appendix comparison tables predate this and need recomputing
Scope of fairness	Full post-merger territory	
Domain	**Discrete zips**, not a continuum	More tractable, not less: exact rather than approximate, and it matches the data

An earlier draft recommended Kalai–Smorodinsky. That was a *continuum* argument (closed-form comparative statics needing only densities, not density derivatives) and does not survive the move to discrete data. It was superseded. KS also systematically awards more absolute gain to the wholesaler who brought the **smaller** book, because `G_b^max > G_a^max` when `S_a > S_b` — defensible in theory, awkward in a room.

---

2. The model in one page

For each postal code `z`: `A_z` (firm A sales), `B_z` (firm B sales), `M_z` (total market opportunity). Totals `S_a`, `S_b`, `M`.

```
c1 = 1 - lam
c2 = theta * (1 - lam)                   # NET headroom

u_a(z) = c1*A_z + c2*B_z + lam*M_z       # value of z to wholesaler a
u_b(z) = c2*A_z + c1*B_z + lam*M_z

g_a(S) = sum_{z in S}  u_a(z)            # gain at d=(0,0): the bundle utility
g_b(S) = sum_{z not in S} u_b(z)

G_a^max = sum_z u_a(z)                   # attainable maxima
G_b^max = sum_z u_b(z)
```

(Earlier drafts subtracted pre-merger books `S_a, S_b` here. That baseline was reversed — `review/HANDOFF_REVIEW.md` §2 — and the code now sets both offsets to zero. An explicit seniority tilt, if distribution insists, goes through asymmetric Nash with a signed-off omega, not through the baseline: `review/code/omega.py::asym_nash`.)

Requires non-negative headroom pointwise: `M_z >= max(A_z + theta*B_z, B_z + theta*A_z)`.

Parameters. `theta` = transfer capture, the fraction of the other rep's book the inheriting rep retains. Estimable from historical territory reassignments by staggered DiD / synthetic control — *this is the one parameter that rests on real identification work and it has not been done*. `lam` = headroom credit, how much a dollar of untapped opportunity counts against a dollar of booked production. Not estimable; it is a compensation-philosophy choice. Reference values used throughout: `theta = 0.40`, `lam = 0.30`.

Two algorithms — use the exact one

Exact (default). `log(g_a*g_b)` is *concave* in the linear expressions `g_a, g_b`, so this is a convex MINLP and outer approximation is a finite global method. Add supporting hyperplanes `z_a <= log(ghat) + (g_a - ghat)/ghat`, maximise `z_a + z_b`, add a tangent at the incumbent, repeat. The tangent set is a relaxation so its optimum is an upper bound; when the incumbent meets it, that is an optimality **certificate**.

Validated against exhaustive enumeration, 120 instances per size: 100% exact match, bound gap identically zero. 6–7 iterations regardless of n — 0.10s at n=50, 0.84s at n=400. `territory.solve(G, zips, "nash")` uses this by default.

Fast heuristic (preview / sanity check).

```
r_z = u_a(z) / u_b(z)                    # exchange rate
sort zips by r_z descending
cumulative-sum u_a and u_b -> g_a[k], g_b[k] for k = 0..n
return argmax_k g_a[k]*g_b[k]  over k with both gains > 0
```

`O(n log n)`, no solver. **This is an approximation, not a theorem** — see trap 1.

---

3. Traps — every one of these was hit during development

1. **The prefix rule is a heuristic, not a theorem — including for Nash.** The usual threshold argument is *marginal*, treating `g_a, g_b` as fixed while a zip moves; discretely they jump by a finite amount. Against exhaustive enumeration it is optimal only ~45% of the time at any size, though the shortfall shrinks with n:

   ```
   n=6   optimal 40%   mean shortfall 3.91%   worst 53.9%
   n=14  optimal 45%   mean shortfall 0.57%   worst 10.7%
   n=50                mean shortfall 0.02%   (vs 1-swap local optimum)
   n=400               mean shortfall 0.0009%
   ```

   It is exact for the *utilitarian* criterion only (`{z : u_a(z) > u_b(z)}`). It fails outright for equal-gain, KS and egalitarian, which minimise a *distance*: over 400k random subsets max-min reached 14.096 vs the best prefix's 13.382. **Use `nash_exact` unless you specifically want a solver-free preview.**

2. **Equalisation can destroy value.** Minimising `|g_a - g_b|` can make *both* wholesalers worse off — an unconstrained equalising MILP hit a gap of 0.000161 at 82% of attainable welfare. Nash cannot do this. If you ever use an equalising criterion, add a welfare floor `g_a + g_b >= kappa * max(g_a + g_b)`; the right-hand side is closed-form because the utilitarian optimum is the prefix `{z : u_a(z) > u_b(z)}`.

3. **The mixture-quantile shortcut is off by one zip on discrete data.** It is exact in a continuum. Do not use it — enumerate the `n+1` prefixes instead.
4. **In the contiguity MILP, the compactness term is not optional.** Fairness alone is degenerate; many allocations tie, so contiguity cuts never bite and cut generation thrashes. An implementation without it ran 17 rounds without converging.
5. **`log(g_a·g_b)` does not linearise the objective — it concavifies it.** That is what makes outer approximation exact (see above). A weighted-sum scalarisation `w*g_a + (1-w)*g_b` **collapses to corner solutions** when the frontier is near linear, which it is. An epsilon-constraint sweep works but needs one MILP per frontier point.

   Set `rho = 0` for exact Nash. A positive compactness weight changes the objective: you are then exactly solving Nash-minus-a-perimeter-penalty, which is a legitimate modelling choice but not exact Nash.
6. **Greedy island repair is catastrophic** — three orders of magnitude worse than the exact MILP. Use it to *detect* fragmentation, never to fix it.
7. **`rho` (compactness weight) competes directly with the objective.** Sweep it; report product and perimeter together. At `rho = 1e-5` the boundary cut 25% of adjacency edges to gain 0.26% of product.
8. **Data uncertainty dominates parameter uncertainty.** Perturbing `lam` and `theta` across their defensible ranges moved 5 of 50 zips; 10% noise on sales moved 34. Getting `A_z` and `B_z` right matters more than settling `lam`.
9. **Nash's axioms assume a convex bargaining set.** Indivisible zips give a point cloud, so `max g_a·g_b` is the discrete analogue rather than the axiomatic solution. Measured cost of not randomising: 0.0034%. Ignore in practice, know it exists.
10. **VOID under `d=(0,0)` — kept for history.** Under the old pre-merger baseline, Nash needed `g_a, g_b > 0` and below a threshold `lambda*` the bargaining set was empty (`lambda*_net = lambda*_gross / (1 + lambda*_gross)`, ~0.069 on test instances). With the zero baseline, gains are bundle utilities — positive by construction — so `lambda*` does not exist and the egalitarian fallback is unnecessary. Demonstrated concretely on the tight-headroom battery case (`battery/FINDINGS.md`, C6): the same instance that was infeasible under the old baseline solves routinely at `d=0`.
11. **Contiguity non-convergence has (at least) three independent causes.** Diagnosed across the C1-C9 battery: (a) pre-existing graph disconnection — the largest free-Nash pair is already split into separate components before contiguity is even imposed (C1-seed2 A0/B0, C5 A2/B2); (b) pure scale — HiGHS time/iteration limits at 125+ zips regardless of topology (C7); (c) value concentration — switching `A_z/B_z` from lognormal to a heavy-tailed (double Pareto-lognormal) draw, same geometry otherwise, **moves which pair fails**: C9-seed2's previously-trivial 31-zip single-component pair (2 iterations under C1) now hits the iteration limit, while C1-seed2's previously-failing disconnected pair now converges cleanly in 16. A fix must be validated against all three, not just the size/topology cases already run. "Decouple fairness from compactness" (solve free Nash, fix a welfare floor, then run a separate perimeter-minimising MILP, mirroring `solve_contiguous`'s KS-gap pattern) was examined structurally and does not address either the disconnection or the scale mechanism — it relocates where the difficulty shows up (one joint MILP to two sequential ones), not whether it exists; it has not been tested against the value-concentration mechanism found in C9.

---

4. Code inventory (`code/`)

File	What it does
`territory.py`	Core module against a networkx ZCTA graph: `validate`, `census`, `overlap_graph`, `zips_for_pair`, `prefix_table`, **`nash_exact`**, `solve` (exact Nash by default; `exact=False` for the heuristic), `compare_criteria`, `contiguity_report`, `contestability`, `write_back`
`districting.py`	Contiguity MILP. `solve_contiguous_nash` = outer approximation + separator cuts. `solve_contiguous` = the KS-gap variant with welfare floor
`territory_demo.py`	End-to-end smoke test on a synthetic lattice with deliberately misaligned territories and state bands
`zip50.py`	Generates the 50-zip worked instance from a Gaussian mixture
`synth.py`	Multi-rep synthetic generator: alignment, correlation, sliver, state, headroom, capacity and (same-day addendum) a heavy-tail dial for `A_z`/`B_z`/`M_z` (double Pareto-lognormal, backward compatible, default off — see `TAIL_DISTRIBUTION_NOTE.md`), each aimed at a named kill criterion; `SCENARIOS` battery including `S7_heavytail`. See `SYNTH.md`
`TAIL_DISTRIBUTION_NOTE.md`	Same-day addendum: what changed in `synth.py`'s tail dial and why (Eeckhout 2004 AER; Reed 2001/2002/2004; Giesen, Zimmermann & Suedekum 2010 JUE), backward-compatibility verification (12/12 PASS), and how to use `S7_heavytail`
`census_stress.py`	Exercises `census()` against the battery: alpha sweep, `min_share` audit, corr-dial check, state binding. Found and motivated the census split patch
`mkfig_census.py`	Figure for the census stress test (`figures/census_stress.png`)
`verify_algebra.py`	35 SymPy checks of every symbolic claim; exits nonzero on failure. Use as a regression test if conventions change. NOTE: standalone symbolic — it does not exercise `territory.py`, so it did not (and could not) catch the `d=0` migration; the numeric anchor for the solver is `review/code/dzero.py`'s self-test
`verify_2d.py`	Reproduces every number in the 2-D note

Expected graph schema

```
G.nodes[z] = {
    "rep_a": <firm A wholesaler id>,   "rep_b": <firm B wholesaler id>,
```

```
    "A": float,  "B": float,  "M": float,
    "state": "NE",                      # optional
}
# edges = Rook adjacency between ZCTAs
```

Minimal usage

```
import networkx as nx, territory as T, districting as D

T.validate(G)                                  # attributes, headroom, islands
for c in T.census(G): print(c["shape"], c["share"])
ra, rb, _ = T.largest_pair(G)
zips = T.zips_for_pair(G, ra, rb)
res  = T.solve(G, zips, "nash")
T.contiguity_report(G, zips, res["to_a"])
cont = D.solve_contiguous_nash(G, zips, rho=2e-3)
T.write_back(G, zips, cont["to_a"], T.contestability(G, zips))
```

---

5. Where to start

Step 1 — the census, before anything else. `T.census(G)` decomposes the national problem into connected components of the bipartite (firm-A rep × firm-B rep) overlap graph. This is gating:

- **1-to-1 pair components** → the two-player theory in `papers/` applies directly.
- **Dense components** (one A rep overlapping several B reps) → two-player fairness does **not** compose. You need leximin over the component, which is not built.

On a synthetic lattice with deliberately misaligned territories the census came back as one dense component covering 100% of opportunity. Expect dense — but note that `census` originally could ONLY answer dense under any boundary noise: its `min_share` trim labelled components without splitting them, so one sliver edge glued clean pairs into a spurious dense blob. **Patched** (`census(..., split=True)`, now the default): trimming re-componentizes, weak edges crossing groups are orphaned, and `1 - sum(share)` is the orphaned opportunity needing adjudication. On real data, report the census at several `min_share` values with the orphan share alongside — the verdict is measurably threshold-sensitive. Evidence and calibration: `SYNTH.md`.

Step 2 — sanity checks on real data, in order of value:

1. `corr(A_z, B_z)` across zips — whether the two books overlap or separate. High positive correlation is the hard case and is likely.
2. Connected components of the resulting allocation on the true Rook graph — price contiguity rather than assuming it.
3. Measurement error on `A_z, B_z, M_z` — this sets which zips need adjudication.

Step 3 — the state-border question. Life and annuity product availability and producer appointments are state-scoped. If territories must respect state lines, that constraint likely dominates contiguity. `respect_state=True` deletes cross-state edges before enforcing connectivity. **Settle this with distribution before tuning anything.**

---

6. Known gaps

- **`theta` is not estimated.** Everything rests on it. Staggered DiD or synthetic control on prior territory reassignments; split by *why* the prior rep left (internal move / industry exit / exit to a competitor — the third is where the loss is).
- **Three or more wholesalers.** No threshold rule exists; leximin over dense components is the natural extension and is unimplemented.
- **Capacity and travel constraints.** Not modelled. A zip's attention cost is probably travel time plus producing-advisor count, not `M_z`.
- **Scale.** `scipy.optimize.milp` re-solves from scratch each cut round. National scale wants a solver with lazy-constraint callbacks (Gurobi, or CBC via PuLP).
- **`contestability()` copies the graph per draw** — fine per pair, slow nationally. Swap for array-based resampling.
- **Contiguity MILP convergence has three known independent failure mechanisms** (graph disconnection, pure scale, value concentration — §3 trap 11) and no fix has been implemented yet. Component-quotient preprocessing, a lazy-callback solver (see the scale bullet above), and a flow-based contiguity formulation are candidates; none has been validated against all three mechanisms.
- **The contiguity loop's two cut families are not the same convergence theory.** The Nash tangent cuts on `log(g_a)+log(g_b)` are genuine Duran-Grossmann outer approximation; the separator cuts enforcing hard contiguity are a distinct combinatorial cut family (closer to branch-and-cut / lazy-constraint generation) bolted onto the same MILP loop. Worth knowing before assuming one convergence argument covers both halves.
- **Optimal transport is not a shortcut here.** The Nash-bargaining/Kantorovich correspondence is real at `d=(0,0)` (M. Warren 2025; cited in `reference/fair_division_2d_20260825.pdf` §10) but adds nothing new for 2 players, and the raw ratio-threshold OT construction does **not** empirically stay contiguous on this project's synthetic data — contradicting that reference paper's own toy example. Not worth pursuing as a contiguity fix.

---

7. Documents

- `papers/nash_territory_division_20260826.pdf` — **the paper.** Nash formulation, prefix property, contiguity MILP, contestability, parameter breakpoints, worked 50-zip instance. Alternative criteria are in its appendices. `.tex` included.
- `reference/discrete_territory_division_20260826.pdf` — earlier draft covering all criteria in the main body. Useful if someone challenges the Nash choice.
- `reference/fair_division_line_20260825.pdf` — continuum theory in 1-D: mixture-quantile closed forms, comparative statics, the monotone-likelihood-ratio regularity condition. Superseded operationally; retained for the analytics.
- `reference/fair_division_2d_20260825.pdf` — 2-D continuum. Contains one result worth knowing: **bimodal books do not disconnect a territory in the plane**, unlike on a line, so part of the apparent contiguity cost in 1-D is a projection artefact.
- `figures/` — the four figures from the paper, plus `census_stress.png` (see `SYNTH.md`).
- `SYNTH.md` — the synthetic battery (`synth.py`) and the census stress test: the census split patch, the `min_share` sensitivity table, and per-kill-criterion scenario instances. Read before running the census on real data.
- `battery/` — the C1–C9 case studies (2026-08-27, C9 added same day): per-scenario figures (pre-merger → exact Nash → contiguous MINLP, each panel a filled zip-level map in `mkfig_census.py`'s style), metrics JSONs, and `battery/FINDINGS.md`. Headlines: rho=2e-3 sits at the knee of the compactness frontier; the census `min_share` threshold moves bookkeeping but not the division; state borders are the binding constraint for multi-state pairs; the contiguity MINLP — not `nash_exact` — is the scale bottleneck (fails on 125+-zip pairs at default budgets); the prefix heuristic is a 0.007%-shortfall screener at 200+

zips; C9 (heavy-tailed sales, same geometry as C1) shows value concentration is a third, independent contiguity risk factor alongside topology and scale (§6, trap 11).

---

8. Opening prompt for the new chat

I am implementing a fair territory-division model in Python for a merger of two annuity wholesaling forces. Read HANDOFF.md first — it contains the settled decisions, the model, a list of traps already discovered, and the code inventory. My data is a networkx graph of ZCTAs with Rook adjacency and per-node firm sales and opportunity. I want to start with [the census / estimating theta / scaling the MILP / …].